Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Web Crawler with Customizable Seed

Student/Team Information

Team Name: Cartographers **Team id.:** DAA-IV-T020

Team member 1 (Team Lead)

- Name: Anunay Sharma
- Student ID: 24021469
- Email: 24021469.geu.ac.in

Team member 2

- Name: Vanshika Kriti Singh
- Student ID: 24022020
- Email: 24022020.geu.ac.in

Team member 3

- Name: Sarthak Singh
- Student ID: 24021521
- Email: 24021521.geu.ac.in

PROJECT PROGRESS DESCRIPTION

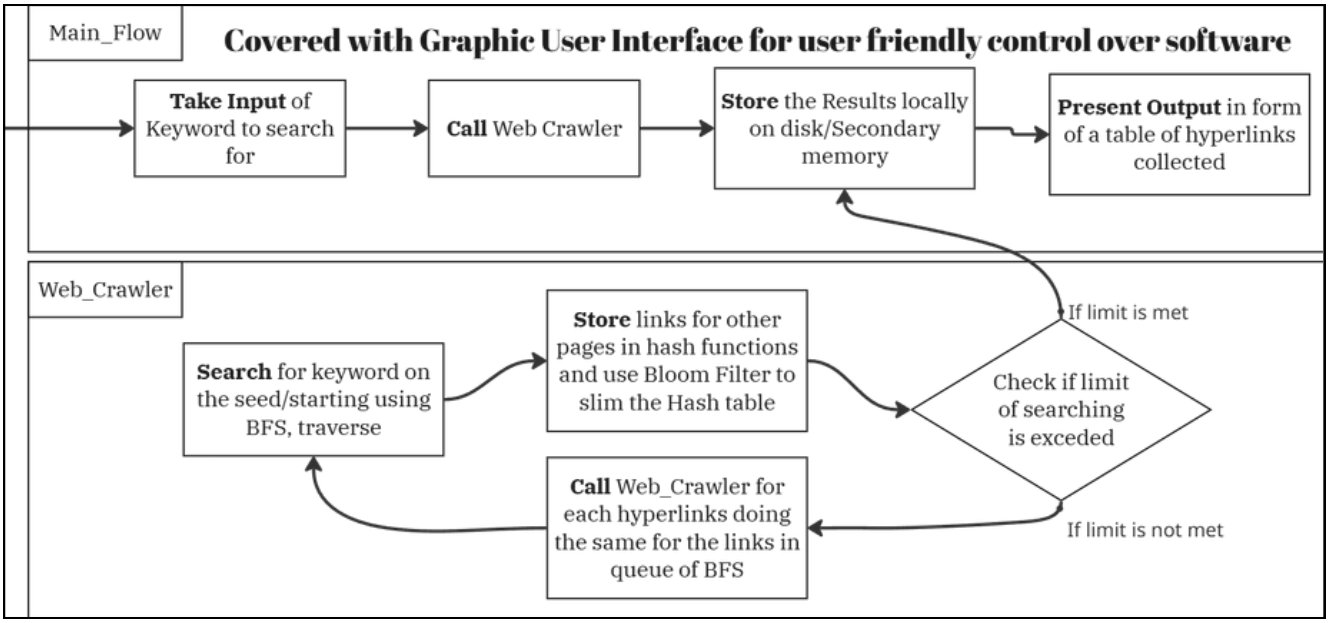
Project Abstract

The main goal of this project is to design a functional web crawler that automates the retrieval of web pages to collect data. The project models the internet as a directed graph where web pages act as nodes and hyperlinks serve as directed edges. This structure naturally maps to a Breadth-First Search (BFS) traversal problem, which is one of the core graph algorithms studied in Design and Analysis of Algorithms.

The crawler accepts a user-supplied seed URL along with a keyword, then systematically explores the web from that starting point. At each step it fetches the HTML content of a page, extracts all outgoing hyperlinks using the JSoup parser, and queues them for future visits. To ensure that the same URL is never processed twice, the project implements Bloom Filters combined with hash-based sets, allowing fast and memory-efficient URL deduplication.

Results are stored locally and presented through a graphical user interface as a formatted hyperlink table. The system ties together graph traversal, probabilistic data structures, HTTP networking, and interface design into one cohesive pipeline.

Updated Project Approach and Architecture



The architecture follows a main flow: keyword input triggers the web crawler, which stores results locally before presenting hyperlinks in tabular format. The crawler module searches using BFS traversal, stores links in hash functions with Bloom Filter optimization to slim the hash table, checks search limits, and recursively processes queued hyperlinks until limits are exceeded.

Tasks Completed

Tasks Completed

Tasks	Team Member
URL Frontier (BFS Queue) + Visited URL Manager (HashSet) — Core data structures for crawl queue and deduplication set implemented and tested	Anunay Sharma
HTTP Fetcher + HTML Parser & Link Extractor — Jsoup-based page fetching with timeout/error handling and anchor tag extraction from fetched HTML pages	Sarthak Singh
BFS Traversal Engine (Core CLI Orchestrator) — Main crawl loop integrating all above modules; end-to-end CLI pipeline functional with depth-limited traversal	Vanshika Kriti Singh

Challenges / Roadblocks

We faced a significant challenge with infinite crawling loops. To avoid this, we plan to strictly limit the depth of our crawling based on the user's needs, ensuring the crawler halts at a user-defined level rather than traversing endlessly.
Managing memory efficiency when storing a massive number of visited URLs is a challenge. We are addressing this by implementing Bloom Filters alongside hash functions to slim the HashSet and efficiently identify unique URLs without storing excessive data, significantly reducing memory overhead.

Tasks Pending

Tasks	Team Member
URL Normaliser + Bloom Filter Integration — Canonical URL formatting and probabilistic deduplication layer using Guava BloomFilter alongside HashSet	Anunay Sharma
Data Storage Module + Priority Scheduler (Heap-based) — CSV/JSON output persistence and PriorityQueue-based URL scoring for relevance-ordered crawling	Sarthak Singh
Robots.txt Compliance Module + GUI (Configuration & Results Interface) — Politeness/crawl-delay handling and full JavaFX/Swing interface with live logs and results table	Vanshika Kriti Singh

Project Outcome / Deliverables

1. A working, end-to-end web crawler driven by a user-supplied seed URL and keyword.
2. A BFS based graph traversal demonstrating queue driven web exploration.
3. A Bloom Filter integrated with a hash set for memory efficient URL deduplication.
4. An HTML parser (JSoup) that cleanly extracts all valid hyperlinks from fetched pages.
5. A data-cleaning pipeline that strips raw HTML and prepares text for downstream use.
6. A GUI to set the seed URL, start the crawl, monitor progress, and export results.
7. Local disk storage of crawled data in a structured format.
8. A final report covering algorithm analysis, complexity discussion, and test results.

Progress Overview

The project is currently ~45% complete. The core CLI pipeline has been successfully implemented and is fully functional — this includes the BFS traversal engine, HTTP fetching via Jsoup, HTML link extraction, URL frontier queue, and visited URL tracking via HashSet. The crawler can be executed from the command line with a seed URL and depth limit, and correctly traverses pages in BFS order while avoiding revisits.

Remaining work includes URL normalisation, Bloom Filter integration for memory-efficient deduplication, a heap-based priority scheduler for relevance-ranked crawling, a robots.txt compliance module, persistent data storage, and the complete GUI interface. The GUI represents the largest single pending effort and will be tackled in the final phase.

Codebase Information

MIRO WHITEBOARD:
https://miro.com/app/board/uXjVJPI27iE=/
GITHUB REPOSITORY:
https://github.com/Bhuknuu/jWebCrawler-Engine/

Deliverables Progress

We have successfully completed several key modules, including a functional, working end-to-end CLI crawler that accepts a seed URL and keyword via the terminal and crawls pages up to a specified depth. The BFS-based graph traversal is fully implemented. Additionally, the HTML parser using Jsoup for hyperlink extraction is complete, ensuring valid http/https links are extracted and filtered correctly. We have also finished the HashSet deduplication portion of our filtering system.

For the modules left to complete, the integration of the Bloom Filter for deduplication is currently in progress. Pending features include the data-cleaning pipeline and storage, as the CSV/JSON output module has yet to be built. Consequently, local disk storage in a structured format is also pending, as it is dependent on that storage module. The GUI for seed input, progress monitoring, and export is planned, with implementation set to begin in the next phase.

Testing and Validation Status

Test Type	Status	Notes
Unit Testing — BFS Queue & HashSet	Passed	Verified FIFO order, correct add/contains behaviour on 50+ URLs
Integration Testing — Fetch + Parse + BFS Loop	Passed	End-to-end CLI crawl tested on wikipedia.org and books.toscrape.com; correct link extraction confirmed
Depth Limit & Loop Prevention Test	Passed	Crawler correctly stops at user-defined depth; no infinite loops observed
Bloom Filter False Positive Rate Test	In Progress	Unit test written; tuning expected insertions and FP rate still ongoing